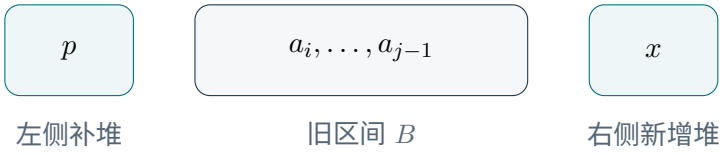
取石子游戏

从必败局面到逆向区间 DP

详细分析 · 完整证明 · 可运行代码

本册围绕你的连续追问整理：不只给出四个分支，
而是解释每个符号、每次回应，以及为什么结论成立。



已知旧区间的两个答案，求新区间唯一的必败补充数量。

阅读后应该能够回答

B 是什么？为什么一个数量不能重复配对？
为什么返回 0、x、$x+1$ 或 $x-1$？
为什么 Case 3 不是循环论证？为什么补 $x+2$ 反而必胜？
为什么计算右侧状态时，`calc()` 的两个状态参数要交换？

版本：2026 年 10 月 20 日 | C++14

包含逐局面策略、区间推演、独立穷举核验与易错点。

阅读路线与目录

先弄清一个原则

“必胜、必败”说的是当前轮到操作的人。
一个局面是必败，并不是说某个固定的人永远会输。每走一步，轮到的人会变化。

内容 **页码**

1. 游戏规则与最基本的胜负递推 3
2. 从两堆、三堆例子建立直觉 4
3. 逆向状态：B、l、r、x、p 分别是什么 6
4. 必败数量的唯一性与存在性 8
5. 配对表：为什么“占用”会影响后面的数量 10
6. 四种返回值与全部边界条件 12
7. 0 和较小的 x：为什么这两种情况成立 13
8. Case 3：$x+1$ 的完整策略与归纳证明 14
9. 为什么 $x+2$ 不行 17
10. Case 4：$x-1$ 的对称证明 18
11. 较大的 x 与零状态 19
12. 右侧状态、函数封装与区间计算顺序 20
13. 样例逐步推演 22
14. C++14 完整实现 23
15. 穷举核验、测试与常见错误 24
16. 自测题、结论速查与参考资料 26

按你的疑问选择入口

符号容易混淆：先读第 3 节。　表格看不懂：重点读第 5 节。

卡在截图的 Case 3：按第 8 节的“左侧、右侧、最小情况”依次阅读。

准备提交代码：读第 12～15 节，但不要跳过边界条件。

本册统一修正的误区

不能直接套普通 Nim 的异或结论；奇数堆不一定必胜；三堆对称也不一定必败；“未被占用”需要最小性与合法操作证明，不能直接等同于必败。

1 游戏规则与胜负递推

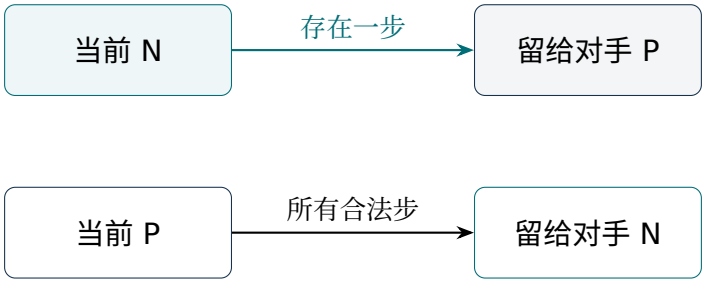
本题每一步只能从**当前最左或最右的一堆**取走至少一颗石子，可以整堆取完。空堆立即消失，新的边界可能暴露出来。不能操作的人输。题目范围为 $n \le 1000$、$a_i \le 10^9$，最多 10 组数据。[1]

先固定“当前玩家”的含义

把局面记为一个有限序列，例如 $(1,2,1)$。定义两类状态：

> **必胜（N）：**存在一步合法操作，把必败局面留给对手。
> **必败（P）：**所有合法操作都会把必胜局面留给对手。

空局面 () 是 P，因为轮到的人无子可取。每一步都严格减少总石子数，游戏不可能无限持续，因此不存在平局；每个局面必属于 N 或 P。



为什么普通的区间胜负数组不够?

如果只写 $win[i][j]$，默认第 i、j 堆仍有原来的数量。但取走一部分后，端点数量会变。例如 $(5,3,2)$ 能变成 $(4,3,2)$、$(3,3,2)$ 等，区间下标相同，局面却不同。每堆可达 10^9，不能把所有剩余数量都枚举进去。

为什么不能用所有堆的异或?

普通 Nim 可以操作任意一堆，本题只能操作两端。例如 $(1,2,1)$ 的异或不为 0，但本题先手必败。**规则变了，胜负公式不能直接迁移。**

本题的突破口

不保存“每种端点剩余数量的胜负”，而是寻找：
固定中间区间后，在一侧补多少颗石子，恰好能让整个局面必败?

2 从两堆、三堆建立直觉

两堆相等必败，不相等必胜

$(3,3)$ 中，后手总在另一堆取相同数量。两堆配对消失，后手拿走最后一颗。$(3,5)$ 则可以一步把较大堆减少到 3：

$$(3,5) \xrightarrow{\text{先手从右侧取 2 颗}} (3,3)$$

所以两堆的结论是：数量相等为 P，不相等为 N。只有一堆正数时，先手直接取完，必为 N。

$(1,2,1)$ 为什么是必败？

先手只能取掉两端的某一个 1。两种选择完全对称：

$$(1,2,1) \xrightarrow{\text{先手取完左端}} (2,1)$$

$$(2,1) \xrightarrow{\text{后手把 2 减少到 1}} (1,1)$$

先手拿一个 1，后手拿另一个 1。于是后手获胜。先手若最初取右端，结论一样。

$(1,2,2)$ 为什么是必胜？

先手只需取完左边的 1，留下 $(2,2)$ 给对手。**证明必胜，只需找到一种获胜操作；证明必败，必须覆盖所有可能操作。**

局面	结果	关键原因
(5)	N	一次取完。
$(3,3)$	P	后手在另一堆对应取子。
$(3,5)$	N	能变成 $(3,3)$。
$(1,2,1)$	P	两种首步均把两堆不相等的局面交给对手。
$(1,2,2)$	N	取掉左端，留下 $(2,2)$。
$(1,1,1)$	N	取掉一端，留下 $(1,1)$。

不要只看奇偶或对称

$(1,2,1)$ 与 $(1,1,1)$ 都是奇数堆、都左右对称，但前者 P、后者 N。因此“奇数堆必胜”与“对称必败”都不是通用结论。偶数堆的完整回文序列可以用成对镜像策略证明必败，但不覆盖全部必败局面。

补充例子：为什么 1 4 1 与 4 1 4 都必败？

$(1,4,1)$：只有两种首步

先手取掉某一端的 1，后手就在剩下的两堆中把 4 减少到 1，留下 $(1,1)$。这个例子只需要两堆结论就能证明。

$(4,1,4)$：不能全程机械镜像

因为左右对称，只需要列出先手拿左端的所有可能：

先手取走	取后局面	后手回应	留给先手
1 颗	$(3,1,4)$	右端取 1 颗	$(3,1,3)$
2 颗	$(2,1,4)$	右端取 2 颗	$(2,1,2)$
3 颗	$(1,1,4)$	取完右端 4 颗	$(1,1)$
4 颗	$(1,4)$	把 4 减少到 1	$(1,1)$

表中还用到了 $(2,1,2)$ 与 $(3,1,3)$ 为 P。可以先验证 $(2,1,2)$：先手在一端拿 1 或 2 颗，后手都能留下 $(1,1)$。

把小例子组织成一个归纳证明

对于整数 $h \geq 2$，命题“$(h,1,h)$ 是 P”成立。假设先手把一端从 h 减少到 z：

剩余 z	后手策略
$z = 0$	剩下 $(1,h)$，把 h 减少到 1，留下 $(1,1)$。
$z = 1$	剩下 $(1,1,h)$，取完另一端 h，留下 $(1,1)$。
$2 \leq z < h$	把另一端也减少到 z，留下更小的 $(z,1,z)$。

基础 $h = 2$ 已证明，其后每次只使用更小的 h，所以归纳成立。$h = 1$ 不在命题中，因为 $(1,1,1)$ 是 N。

为后面准备两个已知答案

固定 $B = (4,1)$：　$(1,B) = (1,4,1)$ 为 P，$(B,4) = (4,1,4)$ 也为 P。
因此旧区间的**左侧答案是 1，右侧答案是 4**。这不是猜测，而是已经由具体策略验证的事实。

3 逆向状态：先把五个符号分清

对输入中的原区间 $[i,j]$，定义：

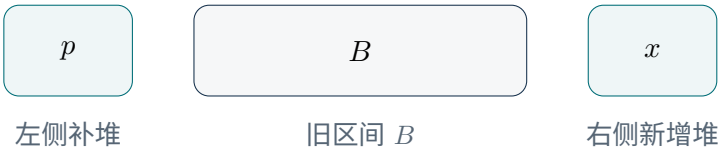
$$L[i][j] = \text{在该区间左边补多少颗，才能使整个局面为 P},$$

$$R[i][j] = \text{在该区间右边补多少颗，才能使整个局面为 P}.$$

两者都允许为 0；0 表示这一侧不增加石子堆。它们记录的是**数量**，不是 0/1 的胜负标记。[2]

计算 $L[i][j]$ 时，统一使用下面的记号

符号	本册中的含义
B	旧区间 $(a_i,\dots,a_{j-1})$，是多堆石子的序列。
l	旧区间的左侧答案 $L[i][j-1]$。
r	旧区间的右侧答案 $R[i][j-1]$。
x	本次纳入新区间的右端一堆，$x=a_j>0$。
p	要求的新答案，即 $L[i][j]$。



这三行的准确含义

已经知道：(l,B) 为 P，(B,r) 为 P。
现在固定右边新增的 x，求左边的 p，使 (p,B,x) 为 P。
旧答案是 l,r，新增数据是 x，新答案是 p。

“固定 B”与“游戏中 B 不可操作”不同

固定 B，只表示**比较不同初始局面时，中间这段不变**。实际游戏中，只要一侧的额外堆被取空，B 的边界就暴露出来，可以合法取石子。竖线只用于分隔，不表示数学运算。

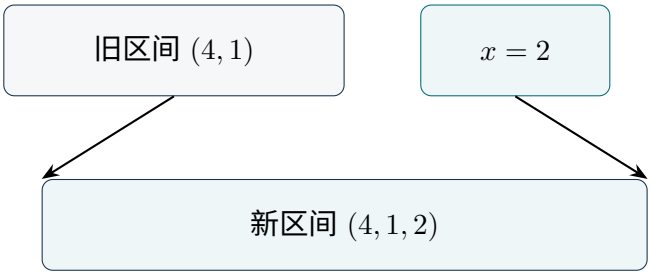
旧答案 l 不等于新答案 p

原题解常把旧答案写作大写 L,R，又用 $L(i,j)$ 表示新答案，容易混淆。本册用小写 l,r 表示旧答案，统一用 p 表示新答案。

从旧区间到新区间：不是在实际游戏里加石子

用 $B=(4,1)$、$x=2$ 走一遍

原来研究的区间只有两堆：$(4,1)$。现在把右端的 2 纳入研究范围，得到新区间 $(4,1,2)$。



再问：新区间左边应该补多少颗?

已知 $l=1, r=4$，最终会算出 $p=3$：旧区间左边需要补 1，新区间左边需要补 3。旧、新答案可以不同。

两个旧结论不能直接拼接

(l,B) 与 (B,r) 是**两个分别成立的局面**，不能推出 (l,B,r) 也必败。本例中：

$$(1,4,1,4) \xrightarrow{\text{取完左端 } 1} (4,1,4).$$

后者是 P，因此前者是 N。

零究竟是什么意思?

$(0,B)$ 与 $(B,0)$ 都只代表 B 自身；不是给游戏增加一堆可以操作的空石子。由定义与唯一性可知：

$$L(B)=0 \iff B \text{ 为 P} \iff R(B)=0.$$

所以旧区间的 l, r 要么**同时为 0**，要么**同时为正数**，不会一正一零。

两种“增加”都属于建模

把输入中的下一堆纳入区间，是计算范围的扩大；想象左边再补一堆，是定义辅助状态。二者都不是允许玩家在游戏过程中增加石子。真实操作只有“取走”。

4 唯一性：为什么补错任何数量都必胜？

首先证明“至多一个”

固定 B，假设有两个不同的补充数量 $u < v$，使 (u, B)、(v, B) 都是 P。可是从 (v, B) 出发，可以在左边取走 $v - u$ 颗：

$$(v, B) \longrightarrow (u, B).$$

这是从一个声称必败的局面，一步走到另一个必败局面，矛盾。$u = 0$ 也成立：直接取空左侧额外堆即可。因此左侧必败数量至多一个，右侧同理。

最关键的博弈事实

P 局面不能一步到达另一个 P 局面。
若能到达，当前玩家就有获胜操作，它应当是 N。

为什么“不能重复使用同一个 p”？

固定 B 和左侧数量 p。若 (p, B, x_1) 已经是 P，且 $x_2 > x_1$，就有：

$$(p, B, x_2) \xrightarrow{\text{右侧取}x_2-x_1\text{ 颗}} (p, B, x_1).$$

所以较大的那个局面必为 N。这里的“占用”只是说**相同的 p 不能再与另一个 x 组成 P**，不是石子真的被某张表拿走。

“只有一个”还需要存在性

上述论证只能得到“至多一个”，不能单独得到“恰有一个”。下一页补全存在性证明。二者结合后，才可以写：

$$p = L(B) \Rightarrow (p, B) \text{ 为 P}; \qquad p \neq L(B) \Rightarrow (p, B) \text{ 为 N}.$$

小于 $L(B)$ 时，为什么也会赢？

必胜不要求一定从左边减少到 $L(B)$；可以操作右侧，改变 B。例如 $B = (3)$，左侧唯一必败数量为 3，但 $(2, 3)$ 可以把右侧减到 2，留下 $(2, 2)$。

不是一道“阈值”题

$L(B)$ 不是“小于就输、大于就赢”的分界线，而是**唯一的必败数量**。此外，题解唯一性段落中的 x 表示左侧补充数量，相当于本册的 p；它与转移中的右侧新增 x 不是同一个角色。

补全存在性：为什么总能找到一个必败数量？

这一页给出一个独立的构造式证明。它用于保证状态定义成立，**不用于实际枚举代码**。

归纳对象：B 自身的总石子数

对总石子数更少的区间，假设左侧必败数量已经存在。空区间的左侧必败数量是 0，因为只有空局面不可操作；任何单独的正数堆都可取完。

若 B 本身为 P，补 0 就完成了证明。下面只讨论 B 本身为 N。

把右侧一步能到达的旧区间列出来

从 B 的最右堆拿一些石子，得到更小的序列 B'。这样的 B' 只有有限种；由归纳假设，每个 B' 都有一个左侧必败数量 $L(B')$。令

$$S = \{L(B') : B' \text{ 是 B 通过一次右侧取子得到的序列}\}.$$

取**不在有限集合 S 中的最小正整数** q。这样的 q 一定存在。下面证明 (q, B) 为 P。

检查 (q, B) 的每一种合法操作

操作右端：得到 (q, B')。因为 $q \notin S$，故 $q \neq L(B')$。结合更小序列的存在性与唯一性，(q, B') 是 N。

操作左端，取完：得到 B。我们正在讨论的前提就是 B 为 N。

操作左端，剩下 $0 < u < q$：由 q 的最小性，$u \in S$。所以存在一个右侧一步可达的 B'，满足 $u = L(B')$。因此

$$(u, B) \longrightarrow (u, B') \quad \text{且后者为 P.}$$

这说明 (u, B) 为 N。

归纳结束

从 (q, B) 出发，所有合法操作都到达 N，因此 (q, B) 是 P。
于是左侧必败数量存在；加上前一页的唯一性，它恰好只有一个。左右翻转后，右侧也完全一样。

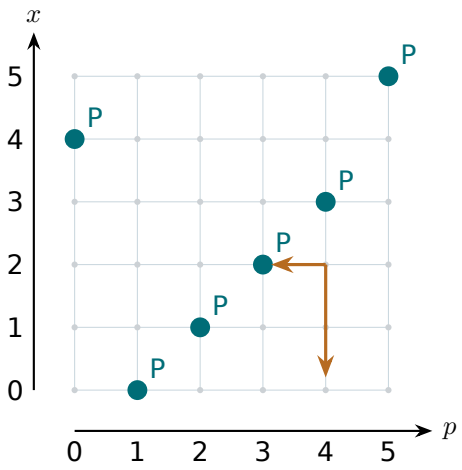
注意，这里对 B 的总石子数归纳，而不是对补上 q 后的总石子数归纳。每个 B' 的总量严格小于 B，归纳方向是合法的。

5 配对表：把两侧数量放进一个坐标网格

固定 B，横坐标是左侧补充数量 p，纵坐标是右侧数量 x。点 (p,x) 代表局面 (p,B,x)；坐标为 0 时，省略对应的额外堆。

内部点的合法移动

当 $p>0,x>0$，最外侧两堆把 B 包在中间。此时每一步只能：减小 p（向左），或者减小 x（向下），可以一直减到 0，不能同时改两边。



例：$B=(4,1)$
$l=1,\ r=4$
绿色点是 P，灰色点是 N。

同一行或同一列，
最多只能有一个 P。
否则较大的点能一步走到较小的 P。

坐标轴不是“也照样递推”

$x=0$ 的一整行由左侧旧答案决定：只有 $p=l$ 为 P。$p=0$ 的一整列由右侧旧答案决定：只有 $x=r$ 为 P。

B 只在内部点暂时被保护

一旦某个额外堆为 0，B 的一端就暴露出来，可能直接被操作。因此坐标轴的胜负应**引用旧区间的已知结论**，不能假装 B 永远不能取。

为什么一定选“最小的未使用正整数”？

令 $f(x)$ 表示这一行唯一的必败左侧数量，即 $(f(x), B, x)$ 为 P。已知 $f(0) = l$；当 $x = r$ 时，$f(x) = 0$。从小到大处理其他正整数 x，把前面各行已经使用过的数量记为

$$U_x = \{f(0), f(1), \ldots, f(x-1)\}.$$

若 $x \neq r$，令 p 为**不在 U_x 中的最小正整数**。证明 (p, B, x) 为 P，必须同时检查两类移动。

向右端取子：右侧变成 $t < x$

如果 (p, B, t) 为 P，就必须有 $p = f(t)$。但 p 没被前面使用过，矛盾。因此这些后继全部为 N。

向左端取子：左侧变成 $q < p$

若 $q = 0$，得到 (B, x)；由于 $x \neq r$，它为 N。

若 $0 < q < p$，由 p 的最小性，q 一定已经被某个旧行 $t < x$ 使用。此时

$$(q, B, x) \longrightarrow (q, B, t), \qquad (q, B, t) \text{ 为 P.}$$

所以 (q, B, x) 为 N。两类移动均不能到 P，故 (p, B, x) 为 P。

“没被使用”还不够，为什么一定要最小?

最小性保证每个更小的正数都已经用过，从而保证“减小左边”也不会一步到达 P。
找到本行的 p 后，任何更大的 q 都能把左侧减少到 p，所以它们必为 N。

你的表格逐行对应到实际局面

x	p	必败局面（省略 0）	为什么是这个数量
0	1	$(1, 4, 1)$	旧答案 $l = 1$。
1	2	$(2, 4, 1, 1)$	1 已用；最小未用正数是 2。
2	3	$(3, 4, 1, 2)$	1、2 已用；下一个是 3。
3	4	$(4, 4, 1, 3)$	1、2、3 已用；下一个是 4。
4	0	$(4, 1, 4)$	正好 $x = r$，B 右补 4 本身已必败。
5	5	$(5, 4, 1, 5)$	1～4 已用，4 那一行不消耗新的正数。
6	6	$(6, 4, 1, 6)$	最小未用正数继续为 6。

6 四种返回值与边界条件

对于**真实新增堆** $x > 0$，新答案为

完整转移公式

$$p = F(x; l, r) = \begin{cases} 0, & x = r, \\ x + 1, & l \le x < r, \\ x - 1, & r < x \le l, \\ x, & \text{其他情况.} \end{cases}$$

本式与公开题解的转移一致[2]；下面用配对顺序和具体获胜回应两条路线展开解释。

从配对顺序读出公式

若 $0 < l < r$**：**第 0 行先使用了正数 l。在到达 l 之前，各行自然匹配 $p = x$；从 $x = l$ 开始必须跳过已经用过的 l，因此 $p = x + 1$。到 $x = r$ 时匹配到 0，少消耗一次正数，之后重新回到 $p = x$。

若 $0 < r < l$**：**第 0 行先保留了较大的 l。$x = r$ 提前匹配到 0，之后正数匹配进度落后一格，所以在 $r < x \le l$ 时，$p = x - 1$；越过 l 后跳过已用的 l，又恢复 $p = x$。

若 $l = r > 0$**：**除了 $x = l = r$ 时 $p = 0$，其余正数行均为 $p = x$。**若** $l = r = 0$**：**第 0 行只使用了 0，所有正数行均为 $p = x$。

三个必须单独核对的边界

条件	正确返回值
$x = r$	一定是 0；不能把它算进 $x + 1$ 或 $x - 1$。
$x = l < r$	$x + 1$，所以加一分支包含左边等号。
$r < x = l$	$x - 1$，所以减一分支包含右边等号。

表格中的 $x = 0$ 与代码不同

配对表用 $x = 0$ 表示“尚未增加右侧堆”，是证明的边界。代码调用 `calc()` 时，x 始终是输入的 $a_i > 0$。不能把四分支公式直接当成所有 $x \ge 0$ 的函数；$x = 0$ 应由 $f(0) = l$ 单独处理。

7 返回 0，以及较小的 x 为什么成立

Case 1：$x = r$ 时，为什么返回 0?

旧答案 r 的定义就是 (B, r) 为 P。现在新增的 x 恰好等于 r，新区间 (B, x) 已经为 P，因此左边无需再补堆，$p = 0$。例如 $B = (4, 1), r = 4$，新区间 $(4, 1, 4)$ 已经必败。

Case 2：$0 < x < \min(l, r)$ 时，为什么 $p = x$?

需要证明 (x, B, x) 为 P。把先手叫甲，后手叫乙。

只要甲没有取空一端,乙就在另一端取同样多,使下一次轮到甲时两端再次相等。设此时两端各有 $h > 0$:

$$(h, B, h) \xrightarrow{\text{甲取左侧}k<h} (h-k, B, h) \xrightarrow{\text{乙取右侧}k} (h-k, B, h-k).$$

两端石子不断减少，甲最终一定先取空一端。此时乙接到的局面是 (y, B) 或 (B, y)，并且

$$0 < y \le x < \min(l, r).$$

于是 $y \neq l$ 且 $y \neq r$。由旧状态的存在性与唯一性，这两个局面对**当前轮到的乙**都是 N。所以乙能赢，原局面即为 P。

镜像到一端被取空，就停止机械模仿

$(2, 5, 2)$：甲取左侧 1，乙取右侧 1，得到 $(1, 5, 1)$。甲再取完左侧后，乙面对 $(5, 1)$，应把 5 减到 1，留下 $(1, 1)$。
若乙继续取完右侧 1，反而会给甲留一堆 5，输掉游戏。

为什么必须要求 $x > 0$?

当 $x = 0$，实际只有 B，不存在两侧的额外堆，上述镜像过程无法开始。例如 $B = (5)$、$l = r = 5$、$x = 0$，实际局面只有一堆 5，是 N，不是 P。

结论不是“对称就必败”

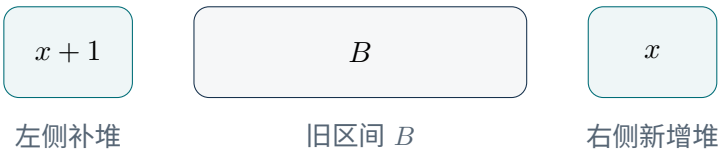
我们用到了两端相等、两端为正、且两端数量都小于 l, r 三件事。$(1, 1, 1)$ 两端也相等，却不满足 $1 < \min(1, 1)$，因此不能套用本结论。

8 Case 3：为什么在 $l \le x < r$ 时返回 $x+1$?

由于 $x > 0$ 且 l, r 同时为 0 或同时为正，这个分支必有 $l, r > 0$。目标是证明

$$P_x = (x+1, B, x) \quad \text{为必败局面.}$$

这里始终对右侧数量 x 从小到大归纳。假设更小的同类局面已经证明为 P。注意：截图中的 z 是**拿完后剩下的数量**，不是拿走的数量。



第一大类：甲拿左端，拿后剩下 z

当前变为 (z, B, x)，其中 $0 \le z \le x$。乙按剩余量分类回应：

条件	乙的回应与合法性	留给甲的依据
$z > l$	把右端 x 减少到 $z-1$。因 $z \le x$，确实至少取 1 颗。	$(z, B, z-1) = P_{z-1}$；$l \le z-1 < x$，是更小同类 P。
$z = l$	取完右端 x。	留下 (l, B)，由旧答案知为 P。
$0 < z < l$	把右端减少到 z；有 $z < l \le x$。	(z, B, z)；$0 < z < \min(l, r)$，由 Case 2 为 P。
$z = 0$	不强求固定回应；乙直接面对 (B, x)，按其获胜策略行事。	$x < r$，所以 $x \ne r$，当前乙面对 N。

两种证明方式不能混淆

前三行是**明确给出乙的一步操作，把 P 交给甲**。最后一行则是**引用旧状态，知道乙当前已处于 N**。两者都能说明乙存在保证获胜的策略。

第一类分支的结论

无论甲从左端取多少，乙要么恢复一个更小的已知 P，要么直接接到一个已知 N。因此甲无法通过左侧首步保证获胜。

Case 3：甲拿右端，以及“递归证明”是否循环

第二大类：甲拿右端，拿后剩下 z

当前变为 $(x+1, B, z)$，其中 $0 \le z < x$。乙回应如下：

条件	乙的回应与合法性	留给甲的依据
$z \ge l$	把左端从 $x+1$ 减少到 $z+1$；因 $z < x$，至少取走 1 颗。	$(z+1, B, z) = P_z$，$l \le z < x$，是更小同类 P。
$0 < z < l$	把左端减少到 z。	(z, B, z)，由 Case 2 为 P。
$z = 0$	把左端减少到 l。因 $x+1 > l$，操作合法。	(l, B)，由旧答案知为 P。

为什么不是“用结论证明结论”？

证明顺序是

$$x = l \implies x = l+1 \implies x = l+2 \implies \cdots \quad (x < r).$$

基础： $x = l$。甲拿左侧后，剩余 $z \le l$，不会进入“$z > l$”分支；甲拿右侧后，剩余 $z < l$，不会进入“$z \ge l$”分支。因此两个引用 Case 3 自身的分支都不发生。其余分支只依赖旧答案或已经证明的 Case 2。

归纳步： $x > l$。左侧拿后的同类参数为 $z - 1 < x$；右侧拿后的同类参数为 $z < x$。每一次只引用更小的 x，不会回到原来的规模。

这是数学归纳法，不是代码递归

证明里出现“递归证明”只是说更大命题依赖更小命题。实际的 `calc()` 只有四个判断，不需要调用自己，也不枚举 $1 \ldots x$。

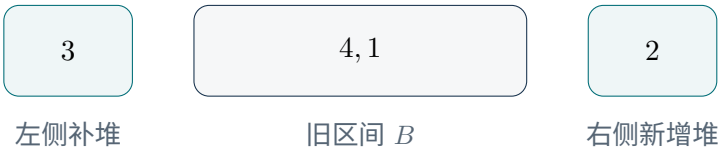
为什么由“后手能应对所有首步”推出 P？

甲从左或右拿，已覆盖全部合法首步；每个首步之后，乙都有获胜办法。所以甲没有保证获胜的首步，原局面 P_x 必败。

结合左侧补充数量的唯一性，能使 (p, B, x) 必败的 p 只能是 $x+1$，不是“大概取一个相近的数”。

Case 3 的完整数字推演：3 | 4 1 | 2

固定 $B=(4,1)$，旧答案 $l=1,r=4$。加入 $x=2$，满足 $1\le 2<4$，所以目标补充数量为 $p=3$。



枚举先手的五种首步

甲的首步	甲操作后	乙的获胜回应
左端取 1	$(2,4,1,2)$	右端取 1，留下 $(2,4,1,1)$，是 $x=1$ 的基础 Case 3。
左端取 2	$(1,4,1,2)$	取完右端，留下 $(1,4,1)$。
左端取 3	$(4,1,2)$	左端取 2，留下 $(2,1,2)$；前面已用全部首步证明它为 P。
右端取 1	$(3,4,1,1)$	左端取 1，留下 $(2,4,1,1)$。
右端取 2	$(3,4,1)$	左端取 2，留下 $(1,4,1)$。

把基础局面 $(2,4,1,1)$ 继续展开

它对应 $x=l=1$，已经到达归纳起点。当前先手只有三种操作：

先手操作	操作后	后手回应
左端取 1	$(1,4,1,1)$	取完右端，留下 $(1,4,1)$。
左端取 2	$(4,1,1)$	取完左端 4，留下 $(1,1)$。
右端取 1	$(2,4,1)$	左端取 1，留下 $(1,4,1)$。

所以 $(2,4,1,1)$ 确实为 P，不再依赖更大的 Case 3。

把抽象证明落到真实操作

大表中的五种首步，每一种都能给出乙的具体回应；小表又补全了递归引用的基础局面。至此，$(3,4,1,2)$ 的必败性有一条完整、可执行的证明链。

9 为什么补 $x+2$ 不行?

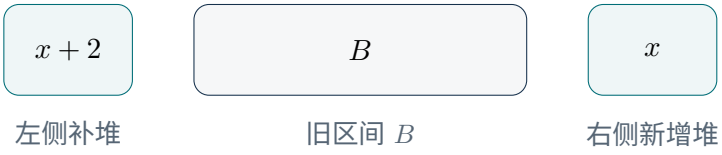
这里“不行”不是说石子数量不合法，而是说它**不能让当前局面必败**。我们要找的是使先手输的补充数量，而不是尽量多补石子。

Case 3 已经证明的结论

在 $l \le x < r$ 下，$(x+1, B, x)$ 是 P。若左边改放 $x+2$，先手立刻可以取走 1 颗：

$$(x+2, B, x) \xrightarrow{\text{先手从左端取 1 颗}} (x+1, B, x).$$

后者轮到对手操作，是 P。因此先手有保证获胜的操作，前者是 N。



左边只需取 1 颗，就把已经证明的 P 留给对手。

代入你熟悉的数字

$B = (4, 1)$、$x = 2$ 时，左边补 3 是 P；补 4 时，先手可以把 4 减少到 3：

$$(4, 4, 1, 2) \longrightarrow (3, 4, 1, 2).$$

比 $x+1$ 大的所有数量都一样

若 $q > x+1$，先手取走 $q - (x+1)$ 颗即可。因此这些 q 全部是 N。

若 $q < x+1$，不能靠增加石子到达它，但仍然为 N。原因来自唯一性，或者配对表的最小未用证明：较小的正数已经匹配过之前某一行，可以减少右端到那一行；$q = 0$ 时由 $x \neq r$ 知 (B, x) 为 N。

三个问题要分开

为什么 $x+1$ 能使局面必败? 需要 Case 3 的全部策略与归纳证明。
为什么 $x+2$ 不能也必败? 因为它能一步减少到已证明的 $x+1$ 局面。
为什么任何其他数量都不行? 因为必败补充数量存在且唯一。

不能仅证明“$x+2$ 会变成 $x+1$”，却省略“$x+1$ 确实是 P”的证明。结论的先后依赖必须清楚。

10 Case 4：为什么在 $r < x \le l$ 时返回 $x-1$？

这一分支要证明 $(x-1, B, x)$ 为 P。最清楚的证明是**把整个游戏左右翻转**，而不是重新背另一套不等式。

翻转后，旧答案也要交换

令 C 为 B 的逆序。左右翻转不改变合法操作与胜负：原来的左端操作变成右端操作，反之亦然。因此

$$L(C) = r, \qquad R(C) = l.$$

令 $y = x - 1$。由于石子数为整数，$r < x \le l$ 等价地给出

$$r \le y < l.$$

这正是 C 上的 Case 3 条件。于是

$$(y+1, C, y) = (x, C, x-1) \quad \text{为 P.}$$

再把整个序列翻转回来，就得到

$$(x-1, B, x) \quad \text{为 P.}$$

这证明 $p = x - 1$，并且同时解释了减一分支为什么是**左边严格、右边含等号**。

用 $B = (1, 4)$、$x = 2$ 验证

由前面的三堆例子，旧答案为 $l = 4, r = 1$。$1 < 2 \le 4$，所以补 $p = 1$，得到 $(1, 1, 4, 2)$。

先手操作	操作后	后手回应
取完左端 1	$(1, 4, 2)$	右端取 1，留下 $(1, 4, 1)$ 为 P。
右端取 1	$(1, 1, 4, 1)$	取完左端 1，留下 $(1, 4, 1)$ 为 P。
取完右端 2	$(1, 1, 4)$	取完右端 4，留下 $(1, 1)$ 为 P。

与 $x+1$ 表格的对应

$B = (1, 4)$ 时，$x = 0, 1, 2, 3, 4, 5$ 对应 $p = 4, 0, 1, 2, 3, 5$。
$x = 1$ 这一行匹配到 0，因此在 $x = 2, 3, 4$ 时，正数匹配进度比 x 少一格。这与翻转证明得到了同一个结论。

11 Case 5：较大的 x 为什么又返回 x?

在 $x > \max(l, r)$ 时，答案重新变成 $p = x$。这一部分用已经严格证明的“最小未用正整数”规律，能够比逐项模拟更清楚地完成证明。

前面所有偏移最终恰好抵消

令 $M = \max(l, r)$。处理完第 $0, 1, \ldots, M$ 行后，已经使用的左侧数量恰好是

$$\{0, 1, 2, \ldots, M\}.$$

为什么？第 0 行用了 l，第 r 行用了 0；其余各行按最小未用正数补齐中间的空位。

因此下一行 $x = M + 1$ 的最小未用正数是 $M + 1$，再下一行是 $M + 2$，依此类推，所以所有 $x > M$ 都有 $p = x$。

用两个配对表对照

右侧 x	0	1	2	3	4	5	6
$l = 1, r = 4$	1	2	3	4	0	5	6
$l = 4, r = 1$	4	0	1	2	3	5	6

两张表在 $x = 4$ 时，都已经用掉 $0, 1, 2, 3, 4$，所以之后的正整数继续按原顺序匹配。

特别情况一：$l = r > 0$

第 0 行用了 l；第 l 行用了 0。其余行没有净偏移，所以除 $x = l$ 返回 0 外，全部返回 x。例如 $B = (3)$：$f(0) = 3, f(1) = 1, f(2) = 2, f(3) = 0, f(4) = 4$。

特别情况二：$l = r = 0$

这说明 B 本身为 P。第 0 行只用了 0，所有正整数行都匹配自身。因此 $x > 0$ 时仍返回 x。例如 B 为 $(2, 2)$，左右各补 1 得到 $(1, 2, 2, 1)$，是偶数堆回文的 P。

不要把 Case 5 偷换成“永久镜像”

较大两端在镜像减少后，可能落进 $x + 1$ 或 $x - 1$ 所在的区间；不能始终认为“相等就必败”。这里的完整依据是前面已证明的配对定理，而不是只凭外观对称。

12 右侧状态、calc 封装与区间计算顺序

把统一转移封装为一个函数

```
int calc(int x, int l, int r) {
    if (x == r) return 0;
    if (l <= x && x < r) return x + 1;
    if (r < x && x <= l) return x - 1;
    return x;
}
```

约定：$x > 0$，l, r 是**同一个旧区间**的左、右必败数量。返回的是“右边加入 x 后，左边应该补多少颗”。

计算左侧状态

旧区间为 $[i, j-1]$，右边加入 a_j：

```
L[i][j] = calc(a[j], L[i][j - 1], R[i][j - 1]);
```

计算右侧状态：把视角翻转

现在旧区间是 $[i+1, j]$，从**左边**纳入 a_i，要求新区间**右边**补多少。把整个区间左右翻转后，原右侧答案成为新左侧答案，原左侧答案成为新右侧答案。因此必须交换传参：

```
R[i][j] = calc(a[i], R[i + 1][j], L[i + 1][j]);
```

要计算	新增 x	函数参数 l	函数参数 r
$L[i][j]$	a_j	$L[i][j-1]$	$R[i][j-1]$
$R[i][j]$	a_i	$R[i+1][j]$	$L[i+1][j]$

名字表达的是“求哪边”，不是“扩哪边”

计算 L 时从右边扩大旧区间；计算 R 时从左边扩大旧区间。L, R 的含义始终是**新区间哪一侧需要补多少颗**。

初始化、计算方向与答案位置

长度为 1 的区间

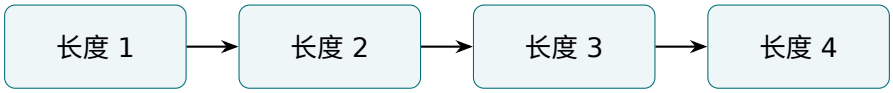
只有一堆 a_i，在另一侧补同样多就形成两堆相等的必败局面，所以

$$L[i][i] = R[i][i] = a_i.$$

按长度从小到大

计算 $L[i][j]$ 依赖区间 $[i, j-1]$；计算 $R[i][j]$ 依赖 $[i+1, j]$。二者都比当前区间短 1，因此使用：

```
for (int len = 2; len <= n; ++len) {
    for (int i = 1; i + len - 1 <= n; ++i) {
        int j = i + len - 1;
        L[i][j] = calc(a[j], L[i][j - 1], R[i][j - 1]);
        R[i][j] = calc(a[i], R[i + 1][j], L[i + 1][j]);
    }
}
```



为什么比较 $L[2][n]$ 和 a_1？

把原来的第一堆看成补在后缀 $(a_2, \ldots, a_n)$ 左边的一堆。后缀的唯一必败补充数量是 $L[2][n]$，因此：

$$a_1 = L[2][n] \iff \text{原局面为 P，输出 } 0.$$

其余情况输出 1。$n = 1$ 时先手直接取完，单独输出 1，避免使用空区间 $[2, 1]$。

另一种等价判断

如果已经算出全区间 $L[1][n]$，那么原局面为 P 当且仅当它等于 0，因此也可以统一输出 `(L[1][n] != 0)`。本册代码保留你之前更熟悉的后缀比较写法。

复杂度与整数范围

单组时间 $O(n^2)$、空间 $O(n^2)$；多组总时间 $O(\sum n_t^2)$。每次调用返回 $0, x, x \pm 1$，其中 x 是原输入值，所以结果至多 $10^9 + 1$，32 位有符号 `int` 足够。这里不计算石子总和。

13 样例逐步推演：3 1 9 4

输入一组数据 $n=4$、$a=(3,1,9,4)$。以下表格列出全部非空区间的状态。

长度	区间	石子序列	$L[i][j]$	$R[i][j]$
1	$[1,1]$	(3)	3	3
1	$[2,2]$	(1)	1	1
1	$[3,3]$	(9)	9	9
1	$[4,4]$	(4)	4	4
2	$[1,2]$	$(3,1)$	1	3
2	$[2,3]$	$(1,9)$	9	1
2	$[3,4]$	$(9,4)$	4	9
3	$[1,3]$	$(3,1,9)$	9	4
3	$[2,4]$	$(1,9,4)$	3	1
4	$[1,4]$	$(3,1,9,4)$	0	0

最关键的两步

先算旧区间 $(1,9)$：

$$L[2][3]=9,\qquad R[2][3]=1.$$

现在右边加入 $x=4$。旧答案是 $l=9, r=1$，满足

$$r<x\le l \quad\Longleftrightarrow\quad 1<4\le 9.$$

所以进入减一分支：

$$L[2][4]=F(4;9,1)=4-1=3.$$

原第一堆 $a_1=3$，恰好等于后缀的唯一必败补充数量，所以原局面 P，输出 0。

两个角度互相检查

后缀比较：$L[2][4]=a_1=3$。
全区间判断：$L[1][4]=R[1][4]=0$。
二者都说明原来的四堆已经必败，外侧不需要再补任何一堆。

14 C++14 完整代码

下面代码与随附的 `stone_game.cpp` 相同。使用标准输入输出；`calc()` 中的左右角色在计算 R 时交换。

```cpp
#include <iostream>
#include <vector>
using namespace std;

// x 是右侧新增堆的石子数，必须大于 0。
// l、r 是旧区间左、右两侧的唯一必败补充数量。
int calc(int x, int l, int r) {
    if (x == r) return 0;
    if (l <= x && x < r) return x + 1;
    if (r < x && x <= l) return x - 1;
    return x;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    if (!(cin >> T)) return 0;

    while (T--) {
        int n;
        cin >> n;
        vector<int> a(n + 1);
        for (int i = 1; i <= n; ++i) cin >> a[i];

        vector<vector<int>> L(n + 2, vector<int>(n + 2, 0));
        vector<vector<int>> R(n + 2, vector<int>(n + 2, 0));

        for (int i = 1; i <= n; ++i) {
            L[i][i] = R[i][i] = a[i];
        }

        // 先计算短区间，再计算长区间。
        for (int len = 2; len <= n; ++len) {
            for (int i = 1; i + len - 1 <= n; ++i) {
                int j = i + len - 1;
                L[i][j] = calc(a[j], L[i][j - 1], R[i][j - 1]);
                // 左右翻转：旧 R、旧 L 分别成为函数的 l、r。
                R[i][j] = calc(a[i], R[i + 1][j], L[i + 1][j]);
            }
        }

        if (n == 1) {
            cout << 1 << '\n';
        } else {
            cout << (L[2][n] == a[1] ? 0 : 1) << '\n';
        }
    }
    return 0;
}
```

15 核验结果与常见错误

本次制作实际做了哪些检查?

检查项目	范围与结果
完整局面穷举	$1 \le n \le 7$，每堆 $1 \ldots 4$，共 21,844 个局面；区间 DP 与独立的记忆化暴力胜负搜索全部一致。
补充数量核验	$1 \le \lvert B \rvert \le 5$、$B_i \in [1,4]$、$x \in [1,6]$、$p \in [0,8]$，共 73,656 组；逐一确认 (p,B,x) 为 P 当且仅当 $p = F(x;l,r)$。
C++14 编译与输出	使用 `g++ -std=c++14 -O2 -Wall -Wextra -pedantic` 编译，21,844 个打乱顺序的局面输出均与暴力一致。
边界与较大输入	另检查 13 组，包括样例、$n=1$、相等/不等两堆、10^9 石子、$n = 999/1000$ 的同值序列。

这是本地验证，不是官方 OJ 提交记录；小规模对拍也不能替代数学证明。它的作用是发现边界、下标、参数顺序等实现错误。

最容易写错的地方

错误写法或想法	修正
把加一分支写成 $l \le x \le r$	右侧必须严格小于 r；$x = r$ 返回 0。
两个调用都按 `L,R` 传参	计算 `R` 时传 `R[i+1][j], L[i+1][j]`。
认为 $L = 0$ 表示“不存在答案”	0 是合法数量，表示区间本身已是 P。
把 z 当作“拿走的数量”	证明里的 z 是“拿完以后剩下的数量”。
只用 `win[i][j]` 保存胜负	部分取子会改变端点数量，普通区间下标不足。
总认为后手必须照着拿	对称策略只在指定阶段使用；端点取空后通常要换策略。

可复现的暴力方法与一组测试

暴力搜索为什么能作为独立参照?

它直接按规则枚举每一步，不使用四分支公式。下面是小数据核验的核心 Python 代码；元组中的堆都是正数，减到 0 就移除该端点。

```
from functools import lru_cache

@lru_cache(None)
def win(a):
    if not a:
        return False

    for z in range(a[0]):
        b = ((z,) + a[1:]) if z else a[1:]
        if not win(b):
            return True

    if len(a) > 1:
        for z in range(a[-1]):
            b = (a[:-1] + (z,)) if z else a[:-1]
            if not win(b):
                return True

    return False
```

可直接复制给正式程序的输入

输入

```
7
1
5
2
3 3
2
3 5
3
1 2 1
3
1 2 2
4
3 1 9 4
4
3 4 1 2
```

输出

```
1
0
1
0
1
0
0
```

各项覆盖

单堆、相等两堆、不等两堆、三堆必败、三堆必胜、题目样例、Case 3 具体局面。

16 自测：先口述，再写公式

六道小检查

1. 固定 $B=(4,1)$，为什么 $l=1,r=4$？这些是原来的石子数量，还是额外需要补的数量？
2. 同一 B 下，$x=3,4,5$ 时，左侧必败数量分别是多少？为什么 $x=4$ 不能使用加一分支？
3. 已知 $(3,4,1,2)$ 为 P，为什么 $(4,4,1,2)$ 为 N？这个论证是否足以独立证明前者为 P？
4. 当 $l=r=0$ 时，为什么正数 x 必须进入最后一个分支，而不是加一或减一？
5. 计算 $R[i][j]$ 时，旧区间是哪一段？为什么传入 R 后再传 L？
6. Case 3 在最小 $x=l$ 时，为什么不会再次引用 Case 3 自身？

参考答案

1. 由 $(1,4,1)$ 与 $(4,1,4)$ 为 P，得到左右补充数量 1 和 4；不是把 B 原来的两堆改掉。

2. 分别是 4、0、5。$x=4=r$ 时 (B,x) 本身已为 P，因此左侧补 0；这是一个独立的边界。

3. 先手在左端取 1 颗即可到 P，故前者以外的较大局面为 N。但必须先独立证明 $(3,4,1,2)$ 为 P，不能把两者循环互证。

4. $x>0$，不等于 $r=0$，也不满足 $0\le x<0$ 或 $0<x\le 0$；此时 B 本身为 P，配对表从 0 之后依次使用每个正整数。

5. 旧区间是 $[i+1,j]$。为了复用“从右边加入”的函数，左右翻转，原右侧答案成为函数里的左侧答案。

6. 左端从 $l+1$ 取后至多剩 l，不会出现 $z>l$；右端从 l 取后严格少于 l，不会出现 $z\ge l$。剩余分支只引用旧状态或 Case 2。

真正应该学会的分析方式

先确定**谁在操作、什么量没有变、什么量变小了**，再谈策略和归纳。不能先背“加一、减一”，再临时寻找理由。

一页速查：从状态含义到代码

核心状态

$L[i][j]$：区间左侧唯一的必败补充数量。
$R[i][j]$：区间右侧唯一的必败补充数量。
0 表示不增加一堆；$L = 0$ 与 $R = 0$ 同时成立。

你问过的问题	最短但完整的答案
B 是什么?	当前已经研究过的连续旧区间，不是一个求和后的大堆。
加入 x，是 B 扩大吗?	是，计算左状态时从 B 扩大为 (B, x)；这是计算范围变化。
原来 $l \mid B$、$B \mid r$?	两个分别已知的 P；不能直接把它们拼成 $l \mid B \mid r$。
为什么 p 不能重复?	相同 p 配两个不同 x，较大的局面能一步减少到较小的 P。
未被使用就必败吗?	还必须是最小未使用正数，并检查左右两类合法后继。
Case 3 为什么不是循环证明?	每次归纳引用都严格减小 x；$x = l$ 是不引用自身的基础。
为什么不放 x+2?	能取走 1 颗到已证明的 $(x + 1, B, x)$，因此反而必胜。
为什么右状态交换参数?	左右翻转后，旧左、旧右的角色互换。

最后只需记住这条推理链

$$\text{P / N 递推} \Longrightarrow \text{唯一的必败补充数量}$$
$$\Longrightarrow \text{固定 B，配对两侧数量} \Longrightarrow \text{四分支转移}$$
$$\Longrightarrow \text{短区间先算} \Longrightarrow L[2][n] \text{ 与} a_1 \text{ 比较.}$$

理解先于记忆

若只记 `calc()` 的四行判断，却说不清 l, r, x 属于哪个区间，最容易在参数顺序、等号和零值上出错。先用 $(3, 1, 9, 4)$ 手算一次，再看代码，会更稳妥。

参考资料与整理说明

资料入口

[1] 洛谷 P2599，**[ZJOI2009] 取石子游戏**。用于核对题意、输入输出和数据范围。

点击打开原题页面

[2] wsyhb，〖**题解**〗**洛谷 P2599 [ZJOI2009] 取石子游戏**，2022 年 2 月 7 日。用于核对逆向状态定义、四分支转移及零值边界；也是本次对话中截图所在的页面。

点击打开本次讨论的题解

本册如何组织证明

本册根据你提出的具体疑问重新编排，不以截图的阅读顺序为唯一顺序。先把符号、P/N 与存在唯一性讲清楚，再用“固定 B 的坐标网格”和“最小未用正整数”推导全部转移，最后展开 Case 2、Case 3 的实际回应以及 Case 4 的翻转证明。

配对定理、构造式存在性论证、具体数字表格与验证程序在本次制作中完整展开。关键结论均给出可检查的推导，不把“题解这样写”当成证明的替代品。

关于核验与适用范围

程序为 C++14 标准输入输出版本，假设输入满足题目范围且各堆初值为正数。`calc()` 的输入 x 是正整数，不能擅自拿证明表格的 $x=0$ 代入。

本文记录的穷举范围与数量来自本次实际执行。数值核验有助于检查实现，不能证明所有大输入；大规模正确性来自正文的状态证明、转移证明及按区间长度的归纳。

复习建议

先把 $B=(4,1)$，$l=1$，$r=4$ 的配对表手写一遍。
然后不看代码，解释：为什么 $x=2$ 时补 3 必败，补 4 必胜；为什么 $x=4$ 时补 0；为什么 $x=5$ 时又补 5。
这四个问题都能说清楚，再写 `calc()` 就不再只是背公式。

整理日期：2026 年 10 月 20 日

本册与随附源代码用于学习、复习和本地验证；未声称已获得官方 OJ 的 AC 记录。